

\1Meccanismo del vettore delle interruzioni. Implementazione del 68k. Esiste la necessità di configurare il vettore. Per poter accedere al vettore esistono diverse cause: **interne e esterne**.

- **Quelle esterne sono quelle per la gestione delle periferiche** che possono essere parallele, seriali, timer. Nel caso del 68 k per problemi di compatibilità sono di due tipi: **vettorizzate o autovettorizzate**. Corrispondono a due configurazioni hardware e 2 configurazioni software.
 - o **Vettorizzate**: Fino a 8 periferiche è la stessa cosa. Se abbiamo più di 8 cause interrompenti possiamo mettere più pic in cascata e quindi più linee di interruzione.
 - o **Autovettorizzato** la richiesta non arriva sulla linea dato, ma arriva sulle IPILO-2. Non è un pic complesso, ma un priority encoder. Il priority encoder dà un numero cioè un livello di interruzione. Il processore accetta se la priorità che interrompe è maggiore dell'interruzione che sta servendo.
- **In un sistema possono accadere più attività: il sistema può avere delle eccezioni interne**, dovute al funzionamento del programma in corso. In questo caso qualcosa comunque si deve gestire. Però è diverso da quello che accade rispetto a quando abbiamo una periferica esterna. In questo caso **è un evento inaspettato** perché chi lo sa che diventerà per 0, oppure un'istruzione non legale. Quindi in questo caso il mapping con la vector table è univoco. Questi sono eventi inaspettati.

Esistono due azioni che sono un po' ibride: **tracce** ho un'interruzione ogni istruzione che viene eseguita, in questo caso non è esterna, non è non voluta, ma è una cosa che non ci aspettiamo. La situazione però è sempre la stessa, vado nella vector table e faccio la ISR.

La printf non è un'eccezione perché io la voglio fare, non è un interrupt perché non viene dall'esterno, quindi ho bisogno di un *codice operativo di passare da stato utente a supervisore*.

Cosa succede normalmente in un sistema

Px è un processo, gli serve la risorsa R1. Se non può prenderla deve fare una call a livello superiore e attivare la ISR.

La risorsa quando deve mandare la risposta, non sa a chi mandarla quindi c'è bisogno di un identificatore di processo.

Px quando fa call da attivo diventa sospeso. Quando ritorna la risorsa da sospeso diventa pronto e non attivo, perché possono esserci più processi.

Senza la system call non possiamo fare un sistema operativo perché non potremmo mai passare da utente a supervisore.

Inizializzazione processo $n=100$

Inizializzazione

Avvio di un carattere

Se non facciamo queste due operazioni la periferica non parte. Queste le facciamo una volta. Dovremmo poter scegliere anche il Modo. L'inizializzazione della periferica non corrisponde all'inizializzazione del processo. Dove facciamo $n=100$.

Se lavorassimo in polling dovremmo sempre vedere il registro di stato e quando si modifica decrementare n .

Non lavoriamo in polling, grazie alla ISR

Quando entriamo nella ISR la prima operazione è gestire il processo

ISR

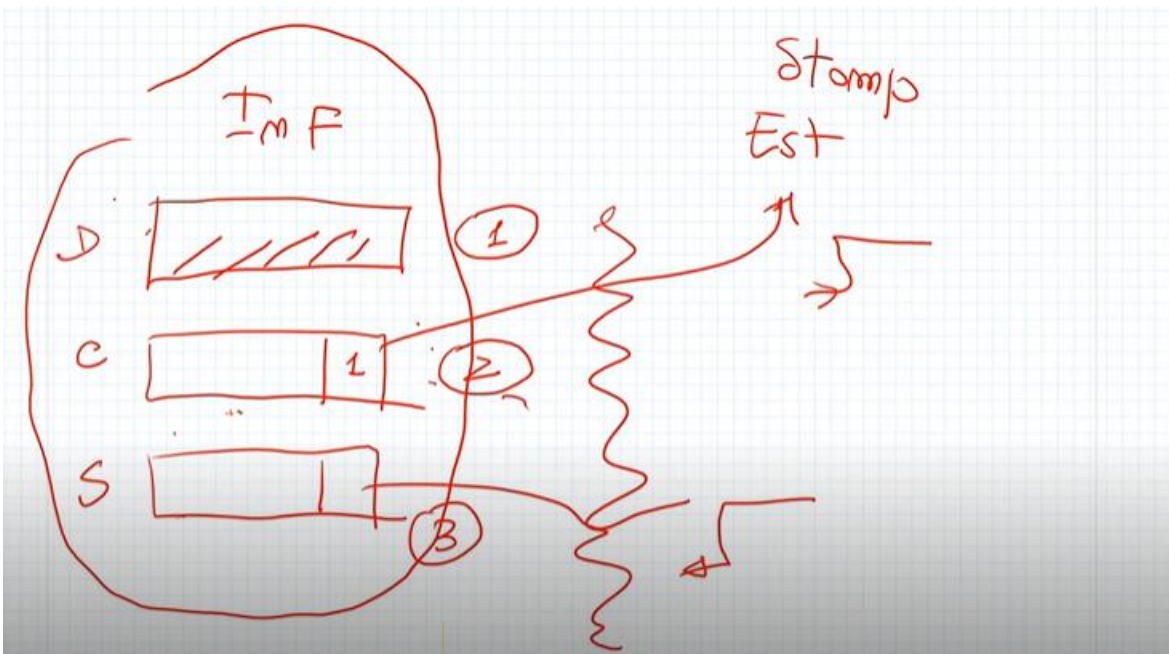
$N=n-1$ gestione processo

Condizione. Devo continuare?

Se si faccio di nuovo Avvio per leggere un altro carattere.

Questa operazione di fatti sembra un ciclo for.

- 1) Ma se inserissi un DMA, cioè un oggetto che ha come compito di gestire la periferica e inviare messaggi sarebbe molto meglio perché avvio il dma e li trasferisce tutti i caratteri. Avrei una sola interruzione ovvero quando il dma ha finito di trasferire. Quindi passo da 100 interruzioni ad 1. Non è detto che serva sempre il DMA. Se devo prendere la posizione di un oggetto non serve il DMA perché è solo un carattere. Se devo misurare una temperatura non serve.
- 2) Se volessi lavorare evitando che quando una periferica mi interrompe che altri mi interrompono avrei due possibilità: **disabilitare le interruzione** e in quel caso nessuno mi può interrompere. Un'altra scelta è quella che **avendo un PIC** possiamo essere selettivi su alcune periferiche. Questo accade se voglio disabilitare periferiche che hanno priorità maggiore, perché se sono inferiori è il processore stesso che non accetta la richiesta.



Ogni volta che vogliamo scrivere un dato scriviamo nel dato. La seconda operazione è scrivere nel controllo che è collegato ad un filo che fa sì che diventi una richiesta per il mondo esterno.

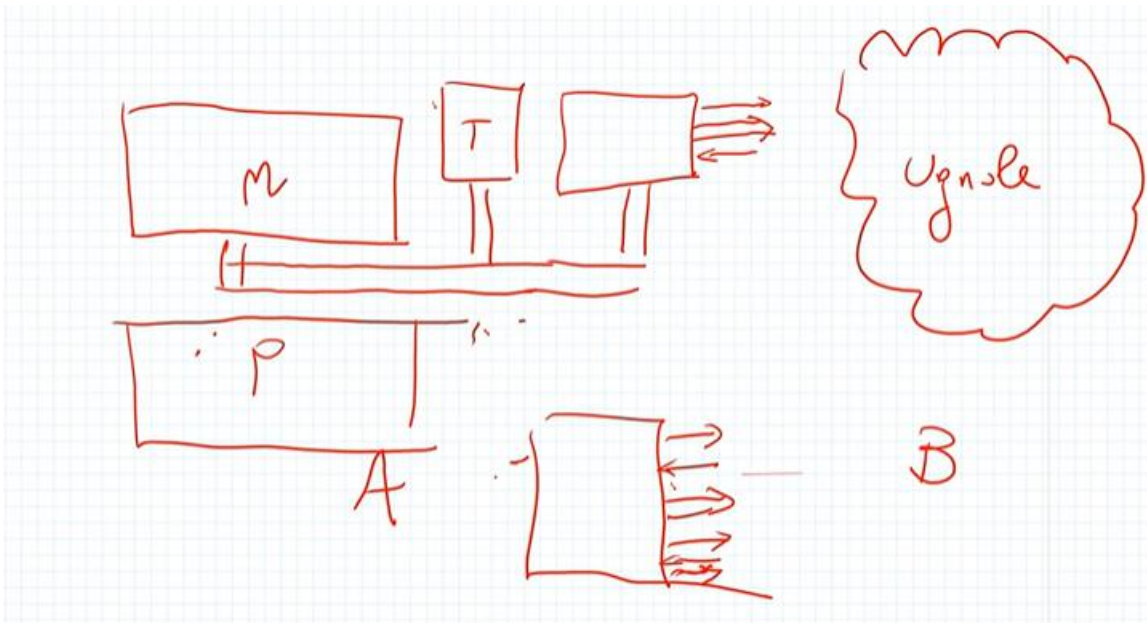
La terza operazione è la ricezione di un'informazione di stato. Il nostro protocollo si basa su queste operazioni.

L'interfaccia si trova nel computer.

L'operazione 1 e 2 avvengono a seguito di aver messo il dato (ipotizzando che sia un'operazione di scrittura).

Il modo di funzionamento deve dire due cose importanti: propaga l'interruzione? Questo segnale è attivo sul fronte di salita o di discesa?

Sul concetto di propagare dobbiamo stare attenti perché se scegliamo di propagare, ma la periferica non propaga allora c'è un mismatch e ci aspettiamo che propaga ma non propaga. Se non propaga dobbiamo fare il polling sul registro di stato.



Abbiamo due nodi gemelli A e B. colleghiamo A e B e faremo sì che si possono mandare messaggi. Quando facciamo questo in Asim ci servono 6 finestre perché una per il processore di A, memoria di A e la periferica di A e idem per B.

L'interfaccia però sarà leggermente diversa perché voglio poter mandare messaggi da A a B e da B ad A quindi l'interfaccia ha controllo stato e dato replicati due volte.

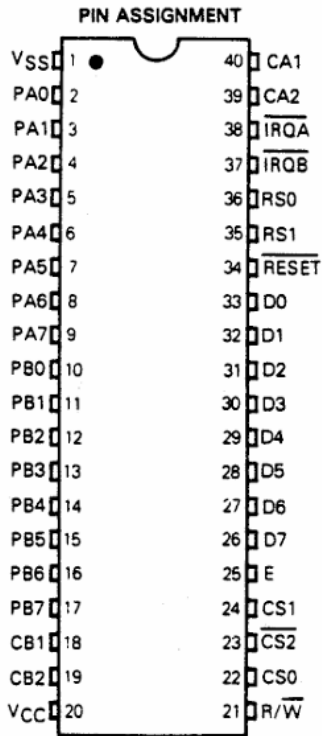
In questo sistema aggiungiamo una periferica: T (Terminal). È comodo perché serve a scrivere il messaggio o vederlo. Quindi alla fine in Asim abbiamo 8 finestre.

PIA

La PIA (Peripheral Interface adapter) è un dispositivo di comunicazione parallela ad 8 bit. Tale architettura è un dispositivo hardware che si posiziona tra la periferica e il processore stesso.

Il sistema ha due canali: un canale A e un canale b verso l'esterno. Per fare il collegamento esterno abbiamo bisogno di fili per l'handshaking come abbiamo sempre detto, quindi due fili per A e due fili per B. poi ci sono 2 fili per le interrupt sempre uno per il canale A e uno per il canale B. Poi ci sono altri fili per l'alimentazione e per collegarsi al bus del sistema o anche bus Interno.

Da D0 a D7 rappresentano il dato che il processore scrive sul BUS sempre in parallelo. Poi è un problema della periferica se mandarlo in parallelo o seriale. In questo caso la PIA lo invierà in parallelo.



Da PA0 a PA7 è il canale A verso l'esterno

Da PB0 a PB7 è il canale B verso l'esterno

CB1/2 sono i fili controllo/stato di B

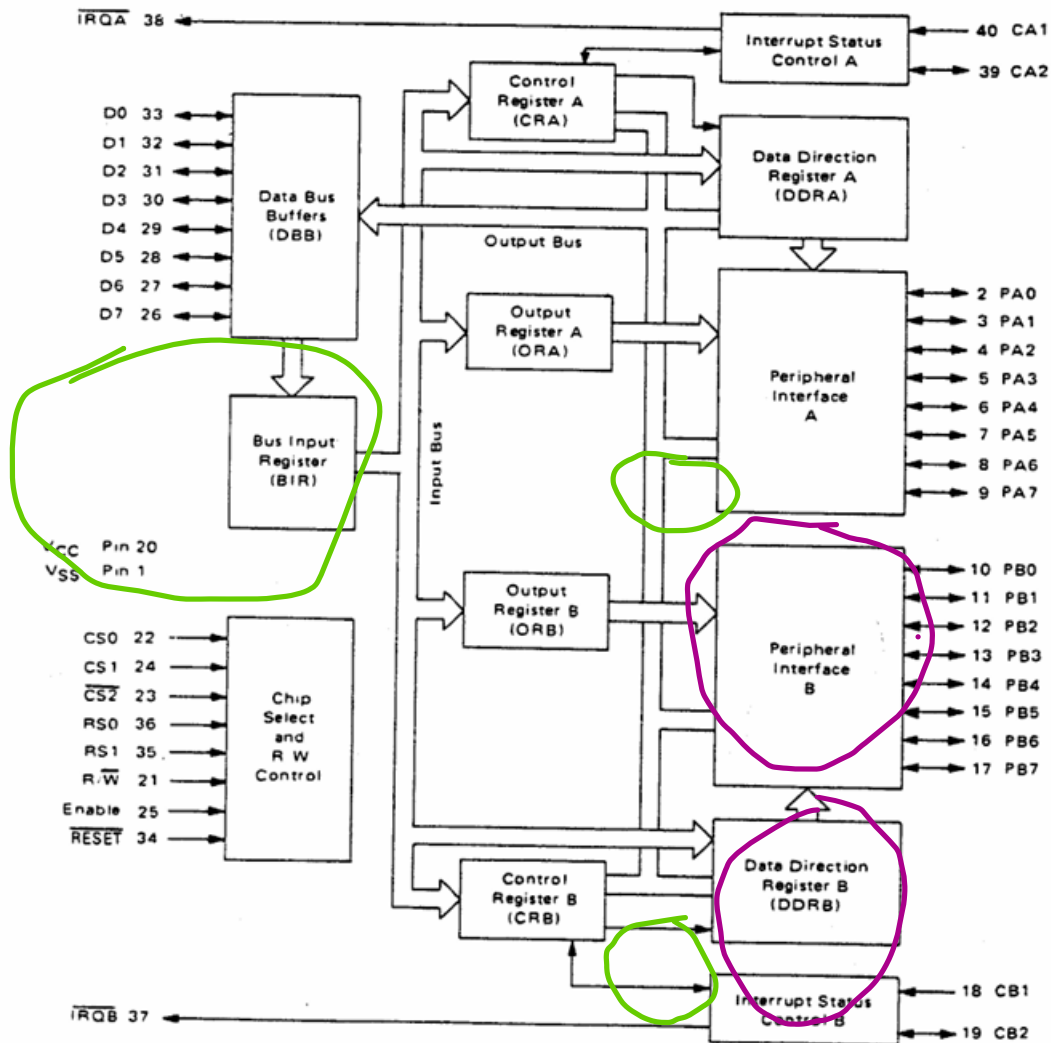
CA1/2 sono i fili controllo/stato di A

IRQA E IRQB sono per le interruzioni di A e di B, vanno a gestire le ISR

D0-D7 collegamento al bus interno

I pin Chip select e R/W servono per selezionare i registri che vede il programmatore

FIGURE 16 – EXPANDED BLOCK DIAGRAM



Questo disegno mostra la PIA nel dettaglio e riusciamo a vedere le due parti che si occupano dei canali A e B verso l'esterno (Peripheral interface A/B). Riusciamo a vedere anche i fili per l'handshacking CA1/2 E CB1/2 anch'essi verso l'esterno e notiamo che CA1/CB1 sono solo di ingresso, mentre CA2/CB2 può essere ingresso e uscita. Questa cosa la settiamo nel registro di controllo del canale. I registri di controllo anche riusciamo ad individuare (CRA/CRB).

A sinistra vediamo ciò che si interfaccia con l'interno e quindi le linee di interruzione e il Dato. Il dato tramite il BUS può andare sia nel canale A che nel canale B.

TABLE 1 – INTERNAL ADDRESSING

RS1	RS0	Control Register Bit		Location Selected
		CRA-2	CRB-2	
0	0	1	X	Peripheral Register A
0	0	0	X	Data Direction Register A
0	1	X	X	Control Register A
1	0	X	1	Peripheral Register B
1	0	X	0	Data Direction Register B
1	1	X	X	Control Register B

X = Don't Care

Possiamo vedere come vengono selezionati i registri. Questa periferica non ha il controllo ha solo il modo e la lettura dello stato. Il controllo è automatico. Ogni volta che scriviamo si attiva l'handshacking.

In realtà il registro si chiama controllo, ma sarebbe il modo perché ci permette di dire se l'interruzioni si propagano, il fronte se di salita o di discesa.

La PIA possiede **6 registri interni** ad 8 bit accessibili (due *Peripheral Registers* per i dati, due *Data Direction Registers* e due *Control Registers*). Tuttavia, riceve dal bus del microprocessore solo due linee per la selezione del registro ($RS0$ e $RS1$). Con due linee, gli stati logici possibili sono solo 4. l'indirizzo finisce nel decoder e 2 bit selezionano i registri, gli altri 30 la periferica

- **RS1 e RS0 (Register Select):** Sono i pin fisici della PIA collegati al bus indirizzi del microprocessore. Definiscono la coppia di indirizzi su cui si sta operando. $RS0$ e $RS1$ sarebbero i bit meno significativi dell'indirizzo a cui stiamo puntando. Se la periferica sta a 8000 significa che 8000 è DDRA/PRA, 8001 è CNTRA, 8002 (bit meno significativi 10) è DDRA/PRA e 8003 (11) è CNTRB.
- **CRA-2 e CRB-2:** Rappresentano il bit 2 rispettivamente del Control Register A e del Control Register B per selezionare direzione/dato nel registro puntato da $RS1-RS0 = 00$.
- **X (Don't Care):** Il valore di quel bit o segnale è influente per determinare il registro selezionato in quella riga.
- **Location Selected:** È il registro che viene effettivamente esposto al bus dati per la lettura o la scrittura.

Operazioni sulla Porta A ($RS1 = 0$)

- Quando $RS0 = 1$, si accede direttamente al **Control Register A**. Gli altri bit vengono ignorati. Questo è l'indirizzo in cui andrai a scrivere la parola di controllo.
- Quando $RS0 = 0$, l'indirizzo è condiviso tra due registri. Per discriminare a quale dei due accedere, l'hardware legge il bit CRA-2:
 - Se **CRA-2 = 1**, le operazioni di lettura/scrittura avvengono sul **Peripheral Register A** (stai manipolando i segnali logici sui pin fisici esterni).
 - Se **CRA-2 = 0**, accedi al **Data Direction Register A** (stai impostando quali pin della porta A faranno da input e quali da output).

Operazioni sulla Porta B ($RS1 = 1$) Il meccanismo è perfettamente simmetrico alla Porta A.

- Quando $RS0 = 1$, accedi sempre al **Control Register B**.
- Quando $RS0 = 0$, l'accesso condiviso è governato dal bit CRB-2:
 - Se **CRB-2 = 1**, accedi al **Peripheral Register B**.
 - Se **CRB-2 = 0**, accedi al **Data Direction Register B**.

A seguito di un reset hardware, la PIA azzerà tutti i suoi registri. Questo significa che i bit CRA-2 e CRB-2 partono forzatamente dal valore 0. Di conseguenza, i primi accessi che il microprocessore farà agli indirizzi con $RS0 = 0$ punteranno in automatico ai *Data Direction Register*, permettendo l'inizializzazione immediata della direzione dei pin.

Quindi le due operazioni fondamentali da fare all'inizio sono dire se il porto scelto è in input o in output perché i fili o acquisiscono dall'esterno o mandano verso l'esterno. Poi dobbiamo dire il modo di funzionamento.

Se abbiamo due nodi per poter comunicare un nodo ascolterà sul canale A e invierà sul canale B. l'altro nodo ascolterà sul canale A e invierà sul nodo B. in questo modo scriviamo un solo driver valido per entrambi i nodi e dovremmo collegare solo porto B del primo nodo al porto A del secondo nodo e porto A del primo nodo al porto B del secondo nodo, quindi poi dovremmo configurare porto A e porto B se sono di input e di output.

Il registro di controllo è composto da dei bit che possono comporre delle macro-aree:

CRA	7	6	5	4	3	2	1	0
	IRQA1	IRQA2	Controllo CA2			Accesso DRA	Controllo CA1	
CRB	7	6	5	4	3	2	1	0
	IRQB1	IRQB2	Controllo CB2			Accesso DRB	Controllo CB1	

La prima volta che scriviamo per decidere la direzione dobbiamo scrivere dentro CRA nel bit numero 2 e mettere a 0 per abilitare l'accesso al registro di direzione. Se mettiamo tutti 1 nel DDR il dato è in uscita, se mettiamo tutti 0 il dato è in ingresso. La successiva volta per scrivere nel dato dobbiamo mettere questo bit a 1.

- **Bit 0 e 1 (Controllo CA1 / Controllo CB1): Gestione degli input di controllo** Le linee CA1 e CB1 sono pin fisici della PIA configurati *esclusivamente* come ingressi. Servono per ricevere segnali dalla periferica esterna. Questi due bit decidono due cose:

questi due bit decidono come comportarsi sui segnali di CA1 e CB1

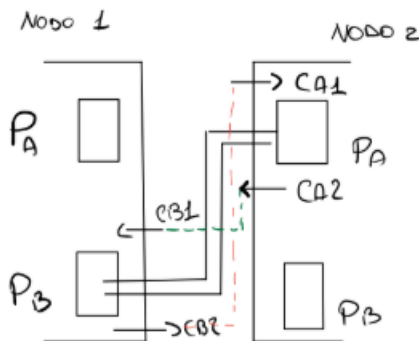
- Se attivare la generazione di un interrupt verso la CPU.
- Su quale fronte del segnale reagire (fronte di discesa o fronte di salita).

Controllano il flag di interruzione IRQA1/IRQB1 presente nella parola al bit numero 7 che si ripercuote sulla linea di interruzione IRQA/B verso il processore.

Nella nostra configurazione i segnali sono attivi nella configurazione da 1 a 0 quindi sul fronte di discesa.

- **CA2/CB2** è differente a seconda che la linea sia stata programmata in entrata o in uscita. Se b5=0 allora si comporta come CA1/CB1 cioè come una linea di ingresso. B3 ha la funzione di b0 e b4 di b1. IRQA2 ha la funzione di IRQA1. Invece Se b5=1 CA2 si comporta come linea di uscita. In tal caso consente di controllare la periferica. Sono previsti 3 possibili modi di sincronizzazione codificati con i bit b4 e b3

Lezione 26 Marzo



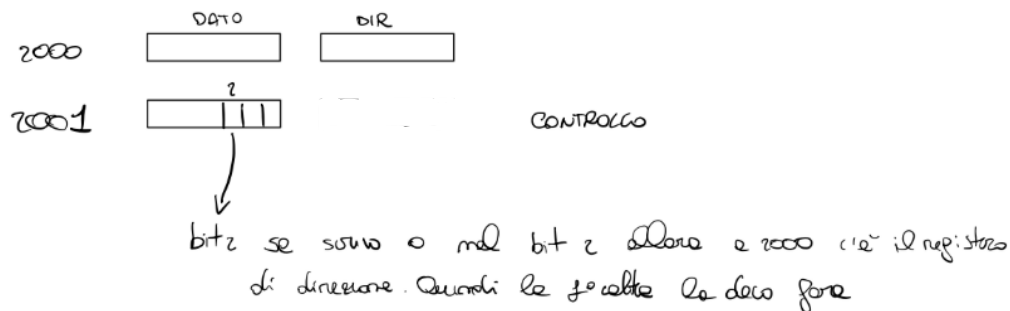
CA1 e CB1 sono i pin
danza se edibili all'ingresso

Architettura e Connessione tra Nodi

- Il sistema è composto da due nodi principali: NODO 1 e NODO 2 che in realtà sono due PIA differenti quindi due sistemi diversi. La PIA è semplicemente l'interfaccia che permette di collegare una periferica del mondo esterno. Ora noi non colleghiamo una periferica ma un'altra PIA. La PIA si trova nel computer, la periferica all'esterno.
- Ogni nodo comunica utilizzando dei porti specifici, denominati P_A e P_B poiché la PIA ha a disposizione due canali.
- Per effettuare correttamente l'handshaking, ogni porto ha bisogno di 2 segnali di controllo dedicati. La PIA ha 4 segnali che permettono ai due porti di fare l'handshaking.
- Nello specifico, i segnali associati ai porti sono CA1 e CA2 (per il porto A) e CB1 e CB2 (per il porto B).
- CA1/CB1 sono adibiti solo all'ingresso, mentre CA2/CB2 se settiamo il bit5 a 1 nel registro di controllo sono adibiti all'uscita.

Registri e Modello di Programmazione

- Per gestire il nodo, devi avere ben chiaro il modello di programmazione: **il modello di programmazione consta nei registri accessibili dal programmatore.**
- Il sistema utilizza dei registri fondamentali: **DATO, DIR (Registro di Direzione) e CONTROLLO.**
- Per ottimizzare e risparmiare spazio, non si usano quattro indirizzi separati, per il dato, direzione, controllo, stato, ma si usano solo due locazioni fisiche. **Nella prima è come se avessimo due registri: Dato e Direzione. nella seconda controllo e stato.**
- La selezione del registro in cui operare avviene tramite un bit di controllo: se vogliamo che l'indirizzo BASE sia la direzione dobbiamo mettere nel bit2 del controllo zero, mentre se vogliamo che all'indirizzo BASE abbiamo il registro dato dobbiamo mettere nel bit 2 uno. Per quanto riguarda controllo/stato, se stiamo scrivendo è il registro controllo, se stiamo leggendo invece è il registro di stato.



Handshaking e Inizializzazione dei Segnali

- 1) La prima operazione da compiere è stabilire i comandi e definire se i porti sono configurati come IN (ingresso) oppure OUT (uscita).
- 2) È necessario **decidere a quali fronti di attivazione i segnali devono reagire**, ad esempio verificando se il segnale passa da 1 a 0 quando si effettua una lettura. Durante la fase di scrittura, **mettiamo il dato nel porto B e il segnale CB2 si attiva in automatico.** Quando il nodo legge il porto A, il segnale lato nodo 2 subisce una **variazione.** Per far sì che i segnali si settino correttamente all'avvio, si deve partire con una operazione di **lettura fittizia.** Si fa sempre.
- 3) Il terzo aspetto è decidere se il sistema lavora in **polling** o con interrupt

Modalità di Funzionamento: Interrupt vs Polling

- I due nodi sono configurabili in modo indipendente: uno può lavorare in interrupt, mentre l'altro può lavorare in polling.
- Le routine di gestione, come le ISR (Interrupt Service Routine), fanno parte integrante del modo di funzionamento del sistema locale.
- **Le ISR sono molto facili da gestire: ogni volta che il sistema riceve un carattere, e lo sa vedendo che il segnale CB1 da alto è diventato basso, scatta un'ISR che lo legge e utilizza un contatore che decrementa per sapere fino a quando dovrà leggere.** Se il trasmettitore invece che in polling fosse in interrupt significa che invia un **carattere e fa altro.** Quando vede che il segnale è cambiato fa partire la ISR per l'invio di un altro carattere e **quindi anche lui avrà un contatore da incrementare per capire quando fermare l'invio.**
- Conoscere lo stato del sistema è di fondamentale importanza per due motivi: è necessario se il sistema lavora in modalità polling, ed è indispensabile se ci sono più periferiche attaccate alle stesse linee di comunicazione.

CRA	7	6	5	4	3	2	1	0
	IRQA1	IRQA2	Controllo CA2		Accesso DRA		Controllo CA1	
CRB	7	6	5	4	3	2	1	0
	IRQB1	IRQB2	Controllo CB2		Accesso DRB		Controllo CB1	

	CRA1 (CRB1)	CRA0 (CRB0)	Flag Interruzione CRA7 (CRB7)	Richiesta Interr IRQA (IRQB)		CRA5 (CRB5)	CRA4 (CRB4)	CRA3 (CRB3)	Flag Interruzione CRA6 (CRB6)	Richiesta Interr IRQA (IRQB)
discesa	0	0	Alto su high/low di CA1 (CB1)	Disabilitata		0	0	0	Alto su high/low di CA2 (CB2)	Disabilitata
	0	1	Alto su high/low di CA1 (CB1) fronte discesa	Inviata quando CRA7 (CRB7) diventa alto		0	0	1	Alto su high/low di CA2 (CB2)	Inviata quando CRA6 (CRB6) diventa alto
salita	1	0	Alto su low/high di CA1 (CB1)	Disabilitata		0	1	0	Alto su low/high di CA2 (CB2)	Disabilitata
	1	1	Alto su low/high di CA1 (CB1)	Inviata quando CRA7 (CRB7) diventa alto		0	1	1	Alto su low/high di CA2 (CB2)	Inviata quando CRA6 (CRB6) diventa alto

Il flag di interruzione CRA7 (CRB7) torna al valore basso in seguito ad un' operazione di lettura su PRA (PRB).

Il flag di interruzione CRA6 (CRB6) torna al valore basso in seguito ad un' operazione di lettura su PRA (PRB).

La tabella a destra è se CRA5=0 e cioè anche la linea CA2/CB2 è di ingresso. Non ci interessa perché a noi sarà sempre di uscita.

Guardiamo i primi due bit meno significativi CRA1 e CRA0 che determinano la configurazione di CA1/CB1:

- Se mettiamo 00 significa interruzione disabilitata
- 01 interruzione abilitata e viene inviata quando CRA7 si alza. CRA7 si alza sul fronte di discesa di CA1
- 10 interrupt disabilitato
- 11 interrupt abilitato e viene inviata quando CRA7 si alza. CRA7 si alza sul fronte di salita di CA1.

Noi lavoriamo sul fronte di discesa quindi settiamo CRA1=0 e CRA0=1. Poi se lavoriamo in polling mettiamo 00.

Notiamo che CRA1 serve per dire fronte discesa(0) /salita (1) e CRA0 serve per dire interrupt disabilitato (0)/ abilitato (1).

Notiamo nella tabella di destra che se il bit5=0, la linea CA2/CB2 è in input e allora CRA4 E CRA3 sono la stessa cosa di CRA1 e CRA0 e quindi con le stesse modalità di funzionamento.

Tabella 6 : Controllo della linea di uscita CA2

CRA5 CRA4 CRA3			CA2	
			BASSO	ALTO
1	0	0	Basso in seguito ad un' operazione di lettura su PRA	Alto quando CRA7 va ad 1 per una variazione h/1 o l/h di CA1
1	0	1	Basso in seguito ad un' operazione di lettura su PRA	Alto al primo colpo di clock successivo alla lettura su PRA
1	1	0	Basso quando CRA3 diviene basso in seguito ad un' operazione di scrittura su CRA	Sempre basso finché CRA3 è basso. Diviene alto se CRA3 va ad 1 per un' op. di scritt. su CRA
1	1	1	Sempre alto finché CRA3 è alto. Diviene basso se CRA3 va a 0 per un' op. di scrittura su CRA	Alto quando CRA3 diviene alto in seguito ad un' operazione di scrittura su CRA

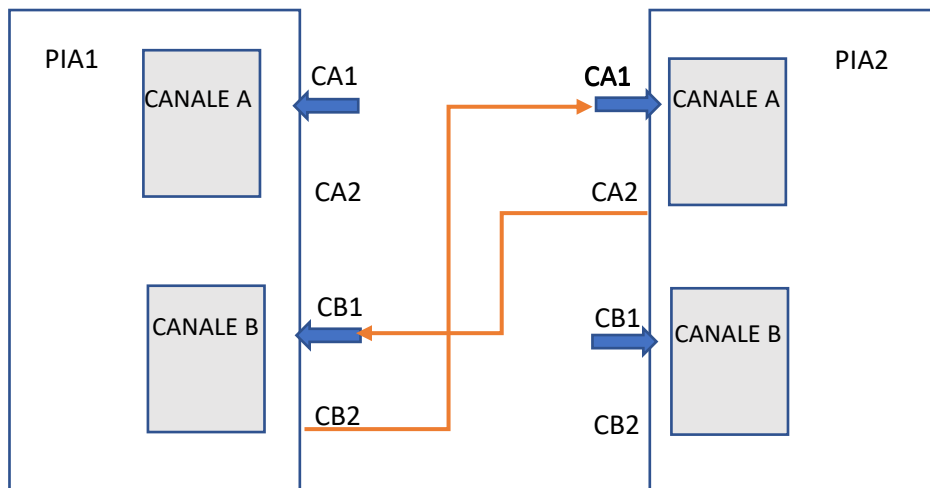
Se CRA5=1 allora CA2/CB2 è settata come linea di uscita. Noi usiamo la configurazione 100. Quindi

CRA5=1, CRA4=0, CRA3=0.

Il che significa che se facciamo un'operazione di lettura CA2 diventa basso. Quindi se leggiamo da PRA/PRB in automatico CA2 diventa basso.

CA2 invece diventa alto appena vede che CRA7 va ad 1. CRA7 diventa 1 sul fronte di discesa di CA1

Il canale A del primo nodo è collegato al canale B del secondo nodo



I due sistemi sono specchiati. Conviene collegare il canale A di un nodo al canale B dell'altro nodo, in modo di dover scrivere soltanto due driver, usandoli in modo specchiato: uno per il canale A e uno per il canale B, riusando gli stessi due nodi, altrimenti dovremmo scriverne 4.

Le linee CA1/CB1 sono solo di ingresso.

Lo stato iniziale dei segnali è CB2 Alto che è collegato fisicamente a CA1, CA2 basso che è collegato fisicamente a CB1

Punto 1: Il Mittente invia il dato

Il microprocessore scrive il byte da inviare nel registro **PRB** del Mittente.

- **Effetto (bit 5-4-3 del registro di controllo B):** L'operazione di scrittura su PRB fa abbassare automaticamente la linea **CB2**. Si alzerà sul prossimo fronte di discesa di CB1
- **Significato:** Il Mittente sta dicendo sul filo: "Ehi Ricevente, ho appena messo un dato nuovo sul bus!".

Punto 2: Il Ricevente viene avvisato

Il segnale basso su CB2 arriva al pin **CA1** del Ricevente perché sono collegati fisicamente.

- **Effetto (bit 1-0 del Controllo A):** Il flag **CRA7** del Ricevente si alza (va a 1). Se l'interrupt è abilitato (CRA0=1), la CPU del Ricevente viene avvisata.
- **Significato:** Il Ricevente capisce che c'è posta per lui.

Punto 3: Il Ricevente preleva il dato

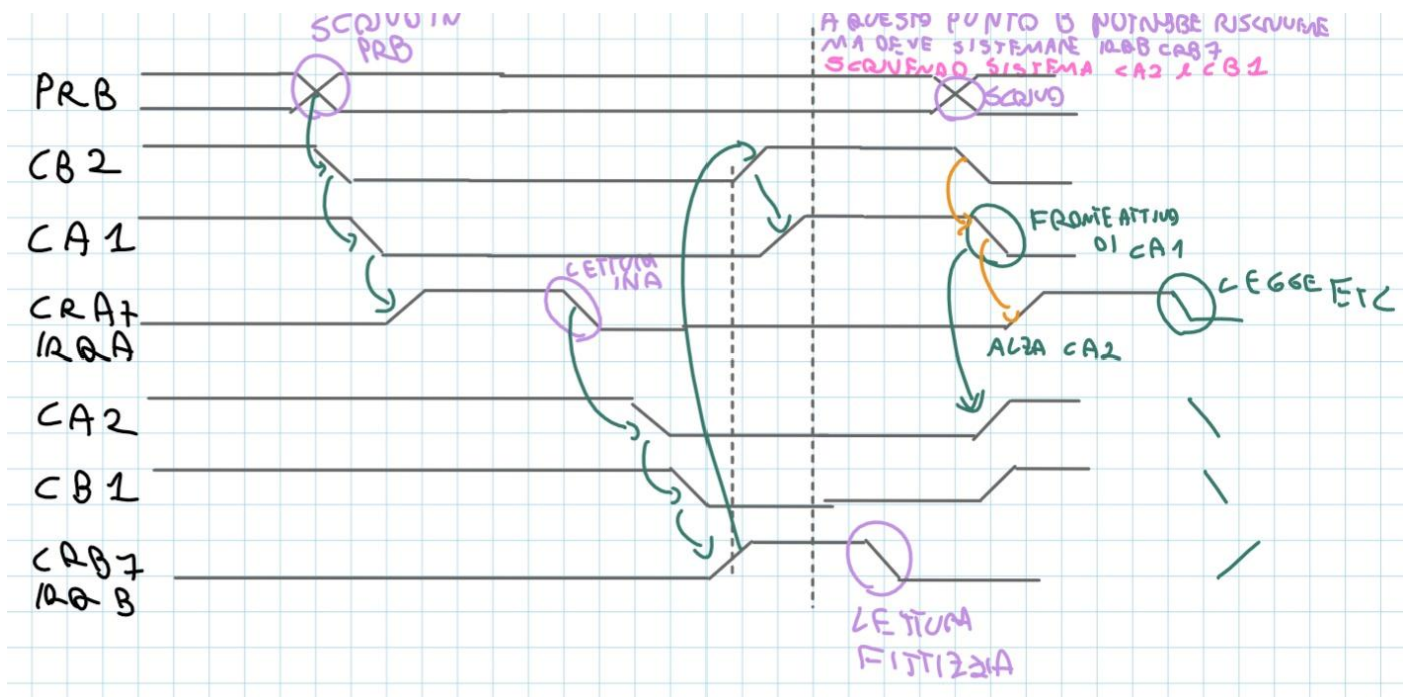
La CPU del Ricevente esegue una operazione di lettura dal registro **PRA**.

- **Effetto (bit 5-4-3 del controllo A):** Leggere PRA fa abbassare automaticamente la linea **CA2** e **CRA7**. Si rialzerà sul prossimo fronte di discesa di CA1
- **Significato:** Il Ricevente dice: "Preso! Ho letto il dato, svuota pure il bus".

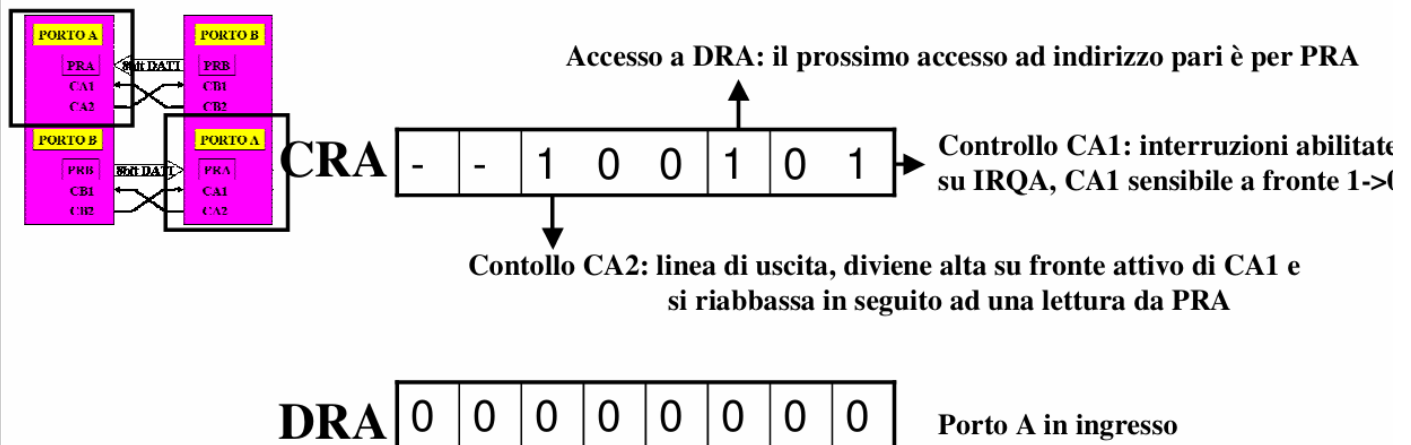
Punto 4: Il Mittente riceve la conferma

Il segnale basso di CA2 arriva al pin **CB1** del Mittente.

- **Effetto (bit 1-0 del controllo B):** Il flag **CRB7** del Mittente si alza (va a 1), e CB2 torna a 1.
- **Significato:** Il Mittente ora sa che il Ricevente è libero. Può procedere a scrivere il prossimo byte in PRB, e il ciclo ricomincia dal Punto 1.



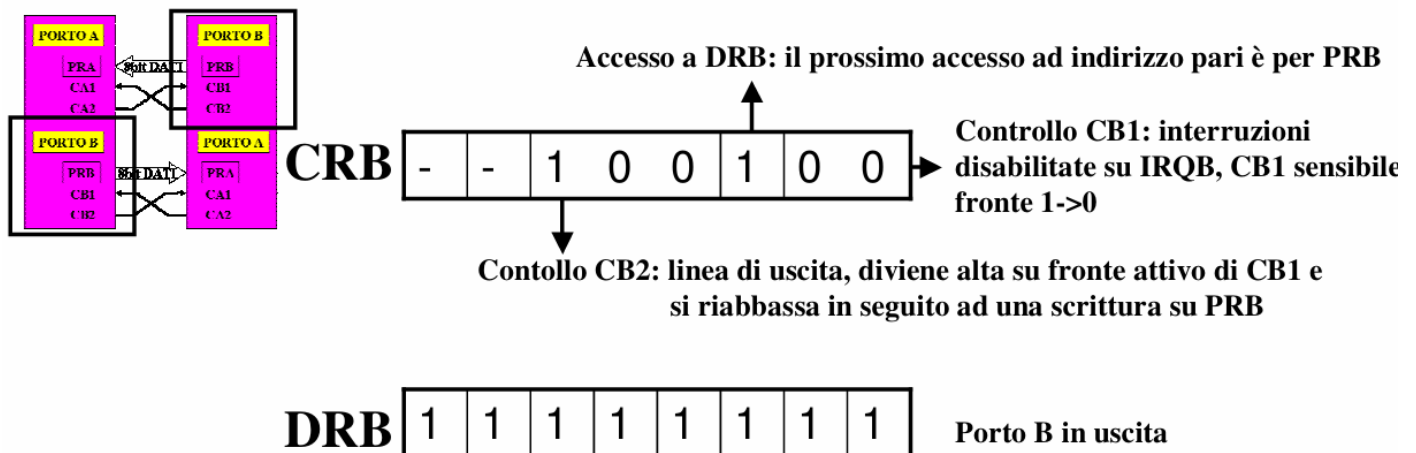
Configurazione Porto A



```

DVAIN  MOVE.B #0,(BASE+1)      ;seleziona il registro direzione del porto A, prossimo
                                     ;accesso ad indirizzo pari =>DRA
        MOVE.B #$00,(BASE)      ;DRA=0 : pone le linee di A a linee di input
        MOVE.B #%11100101,(BASE+1)
        MOVE.B (BASE),D0        ;lettura fittizia per inizializzare CA2 a 0=> CB1 della PIA
                                     ;gemella a 0=>CRB7 sulla PIA gemella a 1
        RTS
    
```

Configurazione Porto B



```

DVBOU  MOVE.B #0,(BASE+3)      ;seleziona il registro direzione di PIA porto B
        MOVE.B #$FF,(BASE+2)   ;pone le linee di PIA B a linee di output
        MOVE.B #%11100100,(BASE+3)
        RTS
    
```

Configurazione Memoria

➤ Livelli d'interruzione associati al terminale:

- » Interruzione per Enter
 - » livello 1, autovettore 25, mappata a \$64, ISR a \$8500
- » Interruzione per Buffer Full
 - » livello 2, autovettore 26, mappata a \$68, ISR a \$8600

➤ Livelli d'interruzione associati alla PIA:

- » Interruzione su linea IRQA - abilitata
 - » livello 3, autovettore 27, mappata a \$6C, ISR \$8700
- » Interruzione su linea IRQB – DISABILITATA
 - » livello 4, autovettore 28, mappata a \$70, ISR \$8700

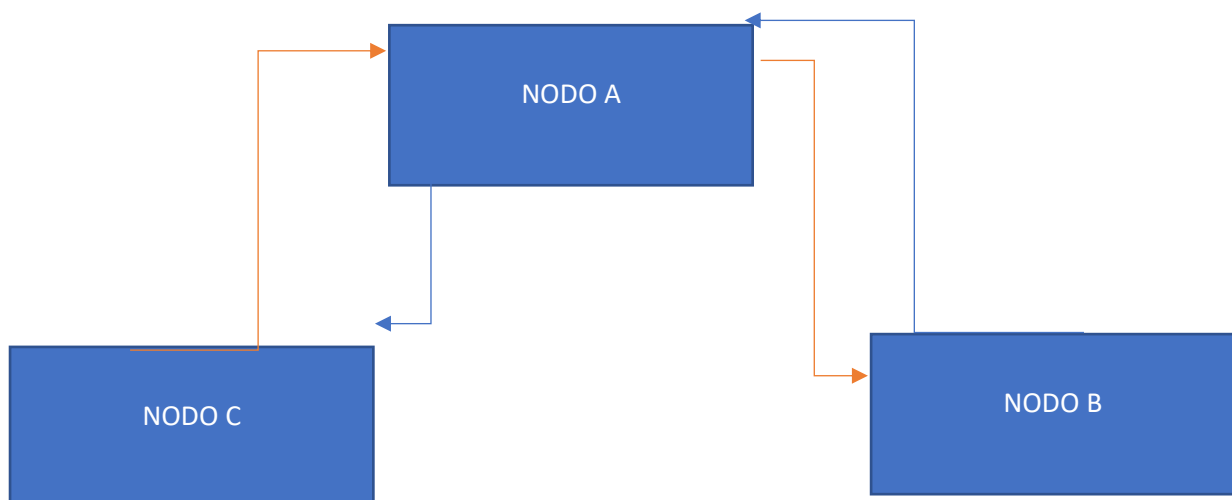
DIS - Dipartimento di Informatica e Sistemistica



Il bit CRA7/CRB7 cioè quello di interrupt si abbassa sempre in seguito ad un'operazione di lettura. Quindi prima di iniziare una comunicazione conviene effettuare una lettura fittizia in modo da ripristinare quel bit.

Lezione 27 Marzo

Nel compito faremo un sistema a 3 nodi



Se inizia B manda tutto B non voglio che venga interrotto da C, e poi posso prendere ancora da B.

Oppure se prendo da B dopo devo prendere da C.

Possiamo fare molte permutazioni. Il problema è di non fare incrociare i messaggi.

Come garantisco che prendo tutti i messaggi da B senza che venga interrotto?

Se arriva prima B il possesso dovrebbe diventare di B. quando cedono il possesso? Quando A ha ricevuto tutti gli N caratteri da B. B non deve perdere il possesso finché non ha mandato tutti i caratteri.

Poss x è una variabile difficile da gestire. Deve essere gestita in mutua esclusione.

Se C è più prioritario di B, arriva C si prende il possesso. Quando B interrompe vede che il possesso è di C e non può andare avanti.

B quando entra nel sistema deve impedire a C di poter entrare pure lui. Quindi entra vede il semaforo e se lo prende. Deve essere un'azione atomica.

Il primo modo per risolvere è che la prima istruzione che fa B o C quando entra nel sistema è di disabilitare le interruzioni. Nessuno lo può interrompere. Poi va a vedere di chi è il possesso. Se è di nessuno se lo prende. Se è di C si ferma. Ha un inconveniente: dobbiamo disabilitare tutte le interruzioni. Funziona come attività SI, ma è una strada molto limitata.

anche le eccezioni interne

L'altra idea è mettere sopra la risorsa un semaforo. Anche se C mi interrompe, vedrà il semaforo rosso e non farà nessun'azione. La prima azione da fare è mettere il semaforo da verde a rosso. Non possiamo usare cmp e set perché sono due istruzioni divisibili. Ci serve un'istruzione atomica. In un solo colpo vede il valore e nel caso setta a rosso.

Istruzione TAS: test and set. Tas testa una variabile ad esempio var e testa il settimo bit di questa variabile. Il semaforo è solo un bit: il bit più significativo di var. se il settimo bit è 0 lo fa diventare 1. Se è 1 lo lascia ad 1.

Se 0 rappresenta il verde quando chiamo TAS entra e mette il semaforo a rosso (1) in modo che chi arriva dopo si ferma.

TAS VAR

BNE FINE

POS? Di chi è?

FINE

Faccio un'operazione di un bit. Il risultato finisce nel bit di stato

Supponiamo che arriva B per primo. Entra va in var e vede il semaforo. Vede che è verde. Lo mette a rosso. A questo punto interroga possesso e non ce l'ha nessuno

Quando B ha inviato il carattere libera il semaforo ma non il possesso.

B è meno prioritario di C. casistiche:

possesso=x

arriva una richiesta di B. va a vedere il semaforo. Il semaforo è a verde. Lo mette a rosso con il TAS. Poiché il semaforo è verde può fare la parte successiva. 2 operazione: B vede possesso. La prima volta non è di nessuno allora possesso = B.

le persone ignoranti fanno il ciclo nella ISR. Le persone intelligenti fanno questa operazione: fanno diventare il semaforo da rosso a verde e termina la ISR (non leva il possesso). Se arriva B un'altra volta, genera un'interruzione. Mette il semaforo a rosso e vede che ha il possesso. Legge il secondo carattere e continua così. Ogni volta che entra ha sempre il possesso. Perde il possesso quando ha terminato di inviare gli N caratteri.

Che succede a C nel frattempo che è più prioritario di B. se arriva quando B ha finito entra mette il semaforo da verde a rosso e vede che il possesso è di B.

Se c arriva mentre B sta nella regione critica (nella ISR). Il semaforo è rosso quindi non vede proprio il possesso.

Quando C fa l'interruzione, chiede che A possa leggere un carattere da B. se non lo posso servire, l'interruzione rimane pendente quindi non può interrompermi più, cioè rimane CRA7/CRB7 sempre alto perché non abbiamo letto. Quando finisco lato B allora se C è fermo quando tolgo il possesso a B lo devo dare a C senno lo blocco all'infinito perché già me l'ha mandata l'interrupt.

- 1) Se voglio che ci sia l'alternanza B C, cioè ricevo tutto da B e poi da C quindi quando B finisce do sempre il possesso a C.
- 2) C. se invece dopo B può tornare da chiunque il possesso quando B finisce non lo do a nessuno.

Avere l'interruzione non significa leggere il carattere.

- 3) Se non mi interessa di prendere prima tutto da B o da C, ma è indifferente prendo un carattere da B uno da C o come capita e li metto in due aeree diverse allora non mi serve la mutua esclusione. La mutua esclusione serve perché vorrei completare prima un messaggio da B o da C. su A abbiamo un conflitto di risorse. O prendiamo da A o da B. abbiamo un'unica aerea. Ci vuole un descrittore di processo (possesso). Per vedere il possesso chi ce l'ha devo essere sicuro di darlo solo ad uno. (è comunista). Mettiamo un semaforo che protegge la variabile possesso in modo che ce l'ha sempre solo una persona. Il semaforo è l'istruzione TAS. la risorsa critica non è il buffer, ma è la variabile possesso. se fosse il buffer, dopo aver scritto un carattere lo rilascio e quindi l'altro può scrivere. invece se è possesso sto prenotando un canale che rilascerò solo al termine
- 4) Se devo partire per forza prima da B significa che possesso lo attribuisco già a B nell'inizializzazione.

Se avessi B C D dopo che B ha finito devo controllare lo stato di C e D per sapere se mi avevano interrotto e sono rimasti sospesi. È qui che conta la priorità!

Quando sblocciamo la periferica pendente dobbiamo sbloccarla alla fine proprio. Perché se sblocciamo prima potrebbe interrompere di nuovo e non ci troviamo. cioè deve essere l'ultima istruzione proprio

Se usassimo il PIC usiamo la maschera.

Se B non genera l'evento, C non genera l'evento come faccio a sapere se qualcuno è rimasto con il braccio alzato e voglio ripulire il sistema. A me serve un evento per vedere se sta qualcuno con il braccio alzato. Mi serve un timer, un generatore di evento. Oppure generare un reset. Ma se non generiamo un evento quel sistema è bloccato all'infinito.

```
# --- ISR B ---

pendingB = 1    # Indico subito che c'è un messaggio/carattere di B da consumare

# 1. Tentativo di acquisizione del controllo del canale
if sender == None:
    if TAS(lock):          # Sezione critica per aggiornare la variabile sender
        if sender == None:
            sender = B      # B prende il controllo del canale
            lock = 0        # Rilascio immediato del lock

# 2. Controllo e Consumazione
```



```

# Si legge il carattere solo se B è effettivamente il mittente corrente
if sender == B and pendingB == 1:
    buffB[curr_ch_B] = ReadPIAB    # Lettura fisica del carattere
    curr_ch_B = curr_ch_B + 1      # Incremento puntatore buffer
    pendingB = 0                   # Carattere consumato con successo

# 3. Gestione fine messaggio e sblocco periferica C
if curr_ch_B == N:                # Se il messaggio di N caratteri è completo
    curr_ch_B = 0                  # Resetta il contatore per il prossimo messaggio
    sender = None                  # Libera il canale

# Logica di Recupero: se C aveva tentato di inviare mentre B era occupato
if pendingC == 1:
    # Prima di sbloccare, preparo le variabili di stato per C
    # (Ad esempio, imposto sender = C e leggo il primo carattere di C)
    sender = C
    pendingC = 0
    readPos = curr_ch_C
    curr_ch_C = curr_ch_C + 1
    buffC[readPos] = ReadPIAC

```

1) Spiegazione doppio if su sender == None:

innanzitutto, eseguire TAS nel caso in cui già sappiamo che sender è diverso da NONE è inutile, stiamo sprecando un codice operativo. Quindi TAS lo facciamo se e solo se non c'è nessuno su sender. Se è none quindi faccio TAS. In questo tempo tra il controllo e TAS, una periferica più prioritaria potrebbe mandare l'interruzione, cambiare sender e prendersi il canale. E rilasciare il lock mantenendo sender occupato. Per questo una volta fatto TAS devo ricontrollare se sender è none.

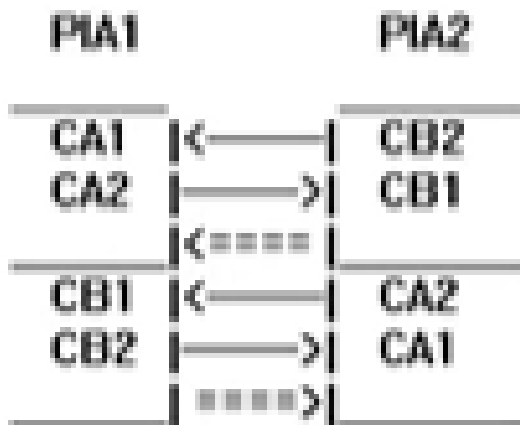
A questo punto se è none occupo il canale e lascio il lock. Perché non ho il rischio di subire l'interruzione tra il secondo if e l'assegnazione? Ho preso il semaforo. Con TAS(lock) ho occupato il lock, quindi se c'è un'interruzione più prioritaria vengo interrotto, farà if (TAS(lock)) ma non entra nella sezione critica perché lock=1 prenotato dall'altra ISR.

2) If sender==B and pending_b==1

A questo punto posso leggere il carattere. Se venissi interrotto non ho problemi perché tanto sending=B quindi non ho rischio di interlacciare i messaggi. Ho il bisogno di fare il doppio controllo, perché se mi trovassi nella ISR e risolvo la situazione di C che mi aveva interrotto nel mentre, quindi termino i miei N caratteri e imposto sending=C effettuando la lettura. Il segnale hardware dell'interruzione potrebbe essere ancora alzato, riportando a leggere nuovamente lo stesso carattere. Quindi facendo il doppio controllo sono sicuro di leggere una sola volta.

3) Recupero di C. il recupero è l'ultima cosa da fare perché se lo facessi prima potrei subire una nuova interruzione da C

Il terminale è un oggetto in cui possiamo scrivere. Quando premiamo ENTER generiamo un'interruzione. Questa interruzione permette di attivare la trasmissione. Dall'altro lato ho la ricezione che butta quel carattere sul video.



In questo sistema entrambi i canali sono collegati in modo che entrambi possono essere sia trasmettitore che ricevitore. Il sistema è simmetrico. In questo sistema noi avremmo 3 indirizzi. Trasmettitore (Canale B), ricevitore (canale A) e terminale.

Per usarlo come ricevitore/trasmettitore devo settare i modi di funzionamento e poi il terminale che è il terzo modo di funzionamento.

Mettiamo il Porto B in uscita. Quindi seleziono il registro di direzione quindi scrivo 0 nel registro PIACB quindi poi scrivo tutti 1 nel registro PIADB quindi uscita. Poi scrivo la parola di 8 bit nel registro PIACB in modo che setta il bit 2 a 1 per avere il registro dato in PIADB e per settare l'handshaking, il fronte di discesa e l'abilitazione dell'interruzione. In questo esercizio lavora sotto polling perché abbiamo messo 00 nei primi due bit, quindi ce l'avrò la ISR della trasmissione? No, avrò solo la ISR della ricezione perché ho messo 01. Quindi quando ricevo ho l'interrupt. Quando trasmetto non ho l'interrupt per sapere se posso trasmettere un altro carattere.

IL TERMINALE è un oggetto che ha un registro di controllo con 8 bit

Cerchiamo di capire come è formato questo registro di controllo del terminale:



Buffer full significa che abbiamo scritto 256 caratteri. Ha insieme controllo e stato. Per attivare il terminale dobbiamo configurarlo. B7 e b6 sono i bit di stato perché diventano 1 se ho il buffer pieno (b6) e se ho premuto enter (b7)

Gli altri tutti a 1 quindi interruzioni abilitate. Se premo enter si abilita l'interruzione

```
BEGIN ORG    $8200

*           Le due configurazioni sono identiche, quindi mi dichiaro gli indirizzi di entrambi i porti qui.
PIADA EQU    $2004 ;indirizzo di PIA-A dato, usato in input
PIACA EQU    $2005 ;indirizzo di PIA-A stato/controllo
PIADB EQU    $2006 ;indirizzo di PIA-B dato, usato in output
PIACB EQU    $2007 ;indirizzo di PIA-B controllo
*           Indirizzo del registro dato e del registro di controllo
TERD EQU    $2000 ;indirizzo di TERMINAL registro dato
TERC EQU    $2001 ;indirizzo di TERMINAL registro stato/controllo
*           Le periferiche vanno tutte e tre inizializzate
JSR    DVAIN ;inizializza PIA porto A
JSR    DVBOU ;inizializza PIA porto B
JSR    DVTER ;inizializza terminal
MOVE.W    SR,D0 ;legge il registro di stato
ANDI.W    #$D8FF,D0 ;maschera per reg stato (stato utente, int abilitati)
MOVE.W    D0,SR ;pone liv int a 000

LOOP JMP LOOP ;ciclo caldo dove il processore attende interrupt
```

È un po' come nei nostri computer: quando lo accendiamo inizializza le periferiche e aspetta che facciamo qualcosa. Quando finisce un'operazione fa LOOP JMP LOOP cioè attesa improduttiva.

Scriviamo Ciao sul terminale e quando diamo enter generiamo l'interruzione. Dove si trova? A 8500. Questo 8500 lo devo mettere nella vector table al livello 1 quindi il 25esimo vector. L'evento è il tasto che viene premuto. Quando il tasto viene premuto noi stiamo dal lato del trasmettitore perché scriviamo per trasmettere. Ma abbiamo detto che lavora in polling quindi viene attivato dall'interrupt dell'enter ma poi dopo fa il polling per trasmettere i singoli caratteri uno dopo l'altro.

premo enter e ho nel buffer la parola "Ciao". questa interrupt si occupa di inviare i caratteri. se la PIA lavora sotto interrupt allora invia solo il primo carattere e gli altri li invia l'interrupt della PIA, se invece lavora in polling allora questa interrupt si mette in attesa sul registro di stato della PIA

Quindi premo enter e parte la ISR. La prima cosa da fare sempre nelle ISR è salvare i registri che sporcherò. Li salvo sullo stack. Poi mi prendo gli indirizzi del terminale e del canale della PIA che uso per trasmettere quindi PIACB e PIADB.

MOVEA.L #TERD, A0 #in A0 c'è l'indirizzo del dato del terminale

cancelletto perchè TERD è un'etichetta \$2000. indirizzamento immediato

Mi prendo il dato del terminale MOVE.B (A0), D0

Poi faccio la lettura fittizia per azzerare CRB7. Poi prende D0 e lo mette sul registro dato del porto B

E a questo punto deve fare il ciclo perché è in polling. Quando il ricevitore risponde e cioè il bit CRB7 è diventato alto deve vedere se ha altri caratteri da inviare o ha finito

Lato trasmettitore ho una sola interruzione.

```

* D1 appoggio
ORG $0500 ricevi da tastiera
INT1 MOVE.L A0,[A7] ;push di A0,A1,A2,D0,D1 in stack supervisor
MOVE.L A1,[A7]
MOVE.L A2,[A7]
MOVE.L D0,[A7]
MOVE.L D1,[A7]
MOVEA.L #TERD,A0
MOVEA.L #PIADB,A1
MOVEA.L #PIACB,A2

INPUT MOVE.B [A0],D0 ;acquisisci dato da terminal

*trasferisci il carattere letto alla PIA-B con handshaking
MOVE.B [A1],D1 ;lettura fittizia => serve per azzerare CRB7 dopo il primo carattere, altrimenti resta settato con l'ack
MOVE.B D0,[A1] ;Dato su bus di PIA porto B: dopo la scrittura di PRB, CB2 si abbassa
;ciò fa abbassare CA1 sulla porta gemella dell'altro sistema generando
;un'interruzione che coincide con il segnale DATA READY

ciclo2 MOVE.B [A2],D1 ;in attesa di DATA ACKNOWLEDGE
ANDI.B #$00,D1 ;aspetta che CRB7 divenga 1
BEQ ciclo2

* fine trasferimento e handshaking
* Posso acquisire vari caratteri, quindi per vedere se ho finito vado a controllare se il carattere successivo da trasmettere e' enter.
CMP.B #13,D0 ;Se il carattere ricevuto e' ENTER
BNE INPUT ;termina altrimenti prossimo carattere
* I 3 bit centrali erano diventati 0, in particolare Tc=11100, ossia non tocco le interruzioni ma pulisco schermo
ORI.B #$1C,TERC ;riabilita tastiera ,pulisce buffer e video
MOVE.L [A7]+,D1 ;ripristino di D0,D1,A2,A1,A0
MOVE.L [A7]+,D0
MOVE.L [A7]+,A2
MOVE.L [A7]+,A1
MOVE.L [A7]+,A0
RTE

```

se il bit di stato è basso la ANDI fa zero quindi rientro nel ciclo. se è alto la ANDI fa 1 ed esco

Quando vedo che si è alzato il bit di stato della PIA devo fare una cosa molto semplice, vedere se devono inviare altri caratteri oppure ho finito. Se il carattere è #13 ho finito perché sta ad indicare enter. Se non è enter allora torna a INPUT altrimenti ho finito. Faccio la OR \$1C con il controllo del terminale.

00011100

RICEVITORE

Ora vediamo lato ricezione che lavora sotto interrupt. Quando riceve il primo carattere scatta l'interruzione. Ho n interruzioni.

La ISR si trova al livello 3 il vettore 27 all'indirizzo 8700.

la prima cosa da fare è usare una maschera per il terminale perché se mentre sto leggendo premo un tasto o enter partirebbe un'interruzione di livello 1 del terminale andando sopra a questa di livello 3.

ANDI.B #\$1101001, TERC mettiamo a 0 interruzioni enter quindi se lo premo non genera interruzione. Poi mette a 0 il bit 5 e quindi disabilita l'input generato dalla tastiera.

E mettiamo a 0 il bit2 così evitiamo che venga resettato lo schermo durante la visualizzazione.

Se è scattata questa interruzione siamo sicuri che c'è un carattere su PIADA quindi prendo questo valore e lo metto sul terminale. La lettura nella PIA ha abbassare CA2 e CRA7. Significa che nell'altro sistema si abbassa CB1 il che corrisponde ad attivare CB7 e quindi potrà mandare un altro carattere. Poi ripristina i registri usati come sempre. Questa routine la faccio n volte, perché è l'interruzione che parte quando CRA7 si alza e manda l'interruzione.

L'ultima istruzione è ORI.B #\$12, TERC. Questa è un OR logico. 00010010 quindi riabilita la tastiera e le interruzioni su enter.

```

    ORG $0700    I
*   Siccome stiamo entrando in una ISR che non e' di peso da altre interruzioni volute dal terminale allora quello che dobbiamo fare e'
*   usare una maschera, quindi metto 0 solo nei posti che voglio disabilitare
*   b1-> interruzioni enter
*   b2-> pulizia schermo ?
*   b5-> tastiera
*   Tutti gli altri bit non ho detto che li disabilto ma che restano come erano prima
INT3 ANDI.B    #$11101001,TERC    ;disabilita: tastiera,cancella video,interruzioni su enter
    MOVE.L    A1,-(A7)            ;salvataggio registri
    MOVE.L    A0,-(A7)
    MOVE.L    D0,-(A7)

    MOVEA.L   #TERD,A0    ;inizializzazione indirizzi device
    MOVEA.L   #PIADA,A1

    MOVE.B    (A1),(A0)          ;acquisisce il carattere e lo trasferisce a Terminal
*   ;la lettura da PRA fa abbassare CRA7 e CA2 => nell'altro sistema si abbassa CB1
*   ;cioè corrisponde all'attivazione di CRB7 che funge da DATA ACKNOWLEDGE

    MOVE.L    (A7)+,D0            ;ripristino registri
    MOVE.L    (A7)+,A0
    MOVE.L    (A7)+,A1
*   * 10010-> b1 rimette interruzioni su enter, b5 riabilita la tastiera
    ORI.B     #$12,TERC          ;riabilita tastiera e interruzioni su enter
    RTE

```

Alcune domande da compito

Se il trasmettitore lavora sotto interruzione il primo carattere lo deve mandare il terminale altrimenti chi attiva questo processo di invio. La routine del terminale.

Se invece della parallela metto la seriale? Solo il codice di inizializzazione. Però se lavora in polling devo cambiare anche il polling perché ho un registro di stato diverso, mentre nella parallela controllo CRA7/CRB7

Se metto il PIC devo cambiare la ISR? La ISR sta sempre a 8700 ma devo cambiare la locazione **perché è vettorizzato**.

Se avessi il DMA l'esercizio di stamattina sarebbe più facile o più difficile? Cioè devo prendere tutto il messaggio da uno senza essere interrotto. Sarebbe molto più facile. perché quello mi interrompe quando ha preso tutto il messaggio. Quindi disabilito l'altro DMA così C non mi può interrompere e prendo tutto il messaggio da B.

Se voglio prendere un carattere da uno e dall'altro con il DMA non lo posso fare perché prende tutto il messaggio.

La periferica sa che sta lavorando con il dma o con il processore? No. Il dma ora si becca tutte le interruzioni che si beccava il processore.

Qual è il conflitto che possono fare processore e DMA? Entrambi vanno in memoria quindi devo avere delle gerarchie di memorie altrimenti devono fare un arbitraggio.

DMA lavora in due modalità: non bloccata il dma ogni volta che mette un carattere chiede al processore di avere il bus. Nella modalità bloccata lo chiedo una sola volta. Il processore non può andare a memoria finché il dma non ha trasferito tutto il messaggio.

Se una periferica è lenta quale modalità conviene? Quella che chiede ogni volta così almeno il processore può fare qualcosa nel mentre.